

Research Assistant API

Design, Implementation, and Evaluation of a Scholarly Discovery Service

Nathan Walsh COMP3011 Web Services and Web Data March 2026

Repository: <https://github.com/NathanWalash/research-assistant-api>

API Documentation (PDF): https://raw.githubusercontent.com/NathanWalash/research-assistant-api/development_branch/docs/Research_Assistant_API_Documentation.pdf

Presentation Slides: <https://docs.google.com/presentation/d/1K8EiXZJHvdmYTVBloDdcs-pwoJtvaxUwbv0WPVq5Gq0/edit?usp=sharing>

Deployed API: <https://research-assistant-api.up.railway.app>

Swagger Docs: <https://research-assistant-api.up.railway.app/docs>

Live Frontend: <https://research-assistant-api.up.railway.app/app>

1 Project Overview

The Research Assistant API is a fully deployed, data-driven scholarly discovery service combining a Python/FastAPI backend, PostgreSQL with pgvector semantic search and citation-graph support, a front-end web interface, and cloud deployment on Railway. It is built on a University of Leeds affiliated subset of the OpenAlex open-access bibliographic database, providing a real corpus of academic papers, authors, topics, institutions, and citation relationships.

The project addresses a practical gap in researcher workflows: existing search tools help discover papers but offer limited support for organising, annotating, and building on what is found. This API brings core research tasks into a single coherent service: corpus discovery, semantic retrieval, citation-graph exploration, personal project management, and personalised recommendation. The platform is intended to be used through several complementary interfaces: a browser-based frontend for interactive use, the Swagger UI at /docs for direct API exploration, a documented MCP interface for AI-assisted workflows (Section 4.4), repository documentation for setup and architecture, and a live Railway deployment for public access.

Beyond baseline CRUD functionality, the system includes vector-embeddings to find similar articles, citation-graph traversal over a derived Leeds-to-Leeds subgraph, a hybrid recommendation engine that blends embedding similarity with citation-proximity scoring, a comprehensive automated test suite covering 13 distinct areas, and a CI pipeline that enforces linting, tests, migration checks, and Docker smoke checks on every push.

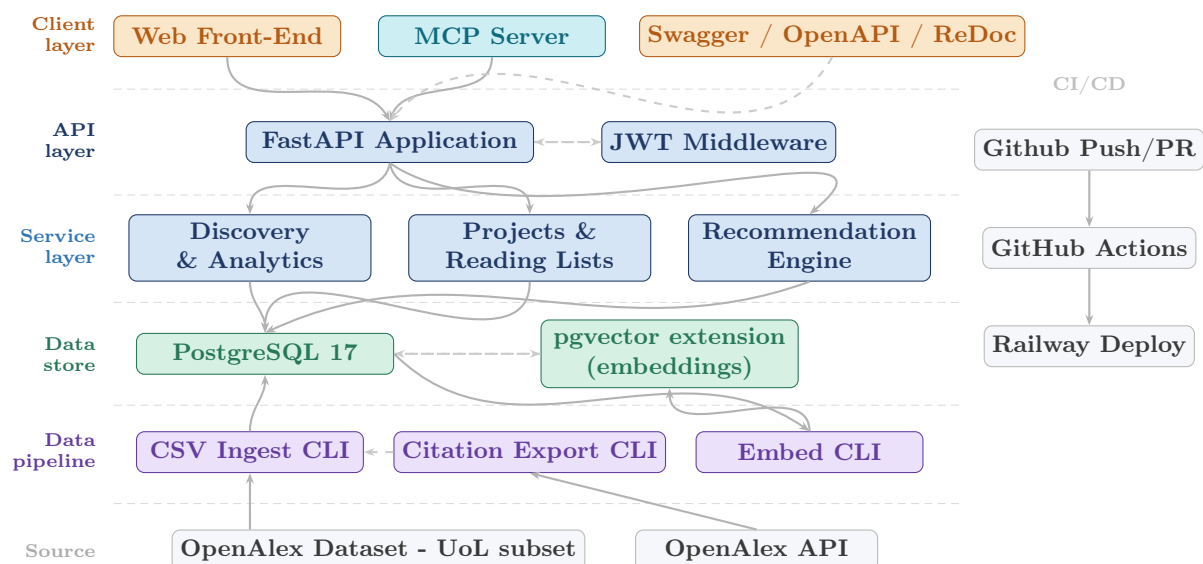
1.1 Development Workflow and Version History

Development followed a branch-per-feature workflow, with `development_branch` used as the primary integration line and focused branches for major subsystems (for example: `dataset-ingestion`, `embeddings-similarity`, `citation-graph-support`, `project-recommendations`, `reading-list-annotations`, `mcp-*`, and `railway-deployment`). This kept changes isolated by capability and reduced integration risk during parallel development.

The repository contains 140 commits, giving a granular implementation trail from service foundation through deployment hardening. This workflow improved reliability and reproducibility by ensuring that major claims in this report (API scope, citation-graph support, embedding-based retrieval, MCP integration, and deployment readiness) correspond to auditable version-history milestones, and by tying branch/commit progression back to a pre-defined project specification and roadmap established before implementation began.

2 System Architecture

2.1 Architecture Diagram



2.2 Stack Rationale and Architecture Notes

Layer	Technology	Rationale
API framework	FastAPI	Async-first with automatic OpenAPI docs, strong request/response validation via Pydantic, and clean dependency injection for route-level services.
Language	Python 3.12	Native fit for data-oriented backend tooling and the project dependencies used across FastAPI, SQLAlchemy, pgvector, and CLI workflows.
Database	PostgreSQL 17 + pgvector	Relational core plus vector search in one datastore; avoids introducing a separate vector database at this project scale.
ORM / migrations	SQLAlchemy 2 + Alembic	Declarative typed models and versioned migrations; <code>alembic upgrade head</code> runs on container startup for schema consistency.
Embeddings	sentence-transformers + pgvector	<code>all-MiniLM-L6-v2</code> embeddings are generated offline by CLI and stored in <code>papers.embedding</code> for low-latency cosine retrieval.
Authentication	JWT bearer tokens	Stateless auth model for REST APIs; bcrypt password hashing via <code>passlib</code> and signed tokens via <code>PyJWT</code> .
Front-end	React + esbuild (served at <code>/app</code>)	Lightweight browser client bundled in-repo and mounted by FastAPI static routes for direct interactive demo access.
MCP interface	FastMCP	Read-only MCP tool surface for papers/authors/topics/analytics/citations plus health checks; mount controlled by environment flags.
Delivery	GitHub Actions + Docker + Railway	CI gates cover API contract, MCP contract, frontend build, Ruff, pytest, PostgreSQL migration validation, and Docker smoke checks; Railway deploy is Dockerfile-based with health checks.

The codebase follows a `src/research_assistant_api` layout, separating API routes, services, repositories, schemas, and ORM models, with dedicated CLI modules for ingestion, embedding generation, citation-graph export, and MCP serving. Core bibliographic entities (`papers`, `authors`, `topics`, `citations`, `institutions`) are separated from user-owned entities (`projects`, `reading_list_items`, `annotations`). Public discovery and analytics endpoints are open; user-specific project/annotation workflows are JWT-protected. Test portability is preserved by using pgvector in PostgreSQL with a SQLite-compatible vector fallback path in tests.

3 Design Rationale and User-Centred Workflow

The system was designed as a research workflow platform rather than isolated CRUD endpoints. The goal was to unify discovery, comparison, note-taking, and shortlisting in one service while preserving a clean API surface. The design was implemented around five principles:

- End-to-end discovery flow.** Public search and citation exploration are open; project, reading-list, annotation, and recommendation actions are authenticated.
- Clear data boundaries.** OpenAlex-derived records are shared corpus data, while projects and notes are user-owned and JWT-scoped.
- Offline enrichment for runtime performance.** Embeddings and citation-edge generation are executed via CLI pipelines, not request-time processing.
- Multiple interfaces over one backend.** The same capabilities are exposed via `/docs`, the frontend at `/app`, and MCP tools.
- Transparent ranking.** Recommendation responses expose component scores and weights for inspectable relevance.

These principles shaped the architecture: service-layer separation, decoupled enrichment pipelines, and deliberate partitioning of public versus authenticated operations.

4 Dataset and Data Pipeline

The dataset is a University of Leeds institution subset from OpenAlex, comprising paper metadata with linked authors, topics, institutions, and aggregate citation counts. Three separate CLI pipelines transform it into a queryable state. First, `research-assistant-ingest` reads the Leeds CSV and populates the relational schema, skipping malformed rows gracefully and reporting a skipped-row count rather than aborting. Second, `research-assistant-embed` encodes each paper's title and abstract with `all-MiniLM-L6-v2` and writes 384-dimensional vectors to the pgvector column. Third, `research-assistant-export-citation-graph` derives explicit Leeds-to-Leeds citation pairs via batched OpenAlex API queries, writing edges incrementally with resumable JSON progress checkpoints so interrupted runs continue from where they stopped.

A key design choice was keeping all three pipelines as separate CLI entry points rather than coupling them to the API startup path, allowing selective re-runs without restarting the service. The derived edge dataset is committed under `data/derived/leeds_citation_edges` so it can be ingested immediately on a fresh deployment. This separation keeps deployment startup lightweight and reproducible, and allows targeted data refreshes (ingest, embeddings, or citation edges) without restarting or modifying the serving API process.

5 API Design and Full Endpoint Reference

5.1 Complete Endpoint Inventory

Method	Endpoint	Auth	Description
<i>Health</i>			
GET	/health	No	Lightweight service health response including status and environment.
<i>Authentication</i>			
POST	/auth/register	No	Create a user account and return a bearer token plus authenticated user payload.
POST	/auth/login	No	Validate credentials and return a bearer token plus authenticated user payload.
GET	/auth/me	Yes	Return the currently authenticated user from the bearer token.
<i>Paper Discovery and Citation Graph</i>			
GET	/papers/search	No	Search papers by free text and optional filters (topic, year, citation threshold).
GET	/papers/{id}	No	Return full paper metadata with linked authors/topics and citation count.
GET	/papers/{id}/similar	No	Return semantically similar papers using stored embeddings.
GET	/papers/{id}/citations	No	Return incoming/outgoing citation neighbourhood in the Leeds citation subgraph.
GET	/papers/{id}/path/{target_id}	No	Return directed shortest citation path between two papers in the Leeds subgraph.
POST	/papers/{id}/annotations	Yes	Create a private annotation for the authenticated user on a paper.
GET	/papers/{id}/annotations	Yes	List the authenticated user's annotations for a paper.
<i>Author Discovery</i>			
GET	/authors	No	List authors in the local Leeds corpus subset.
GET	/authors/search	No	Search authors by name or author ID (query parameter).
GET	/authors/{id}	No	Return author metadata and affiliation details.
GET	/authors/{id}/papers	No	List papers associated with a specific author ID.
<i>Topic Discovery</i>			
GET	/topics	No	List topics present in the local Leeds corpus subset.
GET	/topics/{id}/papers	No	List papers assigned to a specific topic ID.
<i>Corpus Analytics</i>			
GET	/analytics/top-papers	No	Rank papers by citation count with optional topic/year filters.
GET	/analytics/topics	No	Return topic coverage and aggregate citation totals across the corpus.
GET	/analytics/trends	No	Return publication/citation trends grouped by year.
<i>Projects and Reading Lists (Authenticated)</i>			
POST	/projects	Yes	Create a user-owned research project.
GET	/projects	Yes	List all projects owned by the authenticated user.
GET	/projects/{id}	Yes	Retrieve a single user-owned project.
PATCH	/projects/{id}	Yes	Update project title/description (ownership enforced).
DELETE	/projects/{id}	Yes	Delete project and dependent workflow records.
GET	/projects/{id}/recommendations	Yes	Return semantic/citation/hybrid recommendations from reading-list context.
POST	/projects/{id}/reading-list	Yes	Add a paper to a project reading list with optional priority/notes.
GET	/projects/{id}/reading-list	Yes	List reading-list items for a user-owned project.
PATCH	/reading-list-items/{id}	Yes	Update reading-list item priority/notes (owner-scoped).
DELETE	/reading-list-items/{id}	Yes	Delete a reading-list item (owner-scoped).
<i>Annotations (Authenticated)</i>			
GET	/annotations/{id}	Yes	Retrieve a single private annotation owned by the authenticated user.
PATCH	/annotations/{id}	Yes	Update private annotation text (owner-scoped).
DELETE	/annotations/{id}	Yes	Permanently delete a private annotation (owner-scoped).

5.2 Advanced Features Beyond Basic CRUD

(1) Citation-graph enrichment and traversal. The source OpenAlex metadata exposes aggregate `cited_by_count` values but not explicit directed edges. A dedicated export pipeline (**research-assistant-export-citation-graph**) resolves this by deriving Leeds-to-Leeds citation pairs from the OpenAlex API in batches, checkpointing progress to JSON so interrupted runs resume without re-fetching completed work. The resulting edge set is ingested into the local `citations` table, enabling citation-neighbourhood lookup and BFS shortest-path traversal that are not achievable through plain relational filtering alone.

(2) Semantic embedding and vector retrieval. Semantic retrieval is implemented as an offline pipeline rather than request-time inference. The **research-assistant-embed** CLI builds embedding text from paper title + abstract, encodes it with **sentence-transformers/all-MiniLM-L6-v2**, and stores normalized 384-dimensional vectors in `papers.embedding` (pgvector).

These vectors power `/papers/{id}/similar` via cosine-distance ranking in PostgreSQL, while tests retain SQLite portability through a Python cosine fallback path.

(3) Hybrid recommendation pipeline. Project recommendations combine two signals: semantic relevance (embedding similarity to the project reading-list context) and citation connectivity within the Leeds subgraph. Clients can select `mode=semantic`, `mode=citation`, or `mode=hybrid`, with optional `semantic_weight/citation_weight` overrides. Candidate ranking blends both scores and returns a transparent breakdown (`recommendation_score`, `semantic_score`, `citation_score`, and applied weights) for explainable results.

(4) Front-end and live deployment. The system is not backend-only: a React front-end is bundled in-repo and served by FastAPI at `/app`. Deployment uses Railway with Dockerfile-based builds, `/health` health checks from `railway.toml`, and automatic `alembic upgrade head` at container startup so live schema state stays aligned with code. This allows the wider internet to use the API freely.

(5) MCP server integration. The project includes a Model Context Protocol interface for AI-assisted discovery workflows. The MCP surface is intentionally read-only and focused on public operations (papers, authors, topics, analytics, citation neighbourhood/path), with authenticated mutation routes excluded by design. It can run in stdio mode or as an HTTP mount (for example `/mcp`), allowing assistants to invoke research operations through structured tools instead of manual endpoint composition.

6 Testing, Reliability, and Validity

6.1 Testing Strategy

127 tests were incorporated across 42 test modules to cover all expected behaviours, edge cases, and failure modes.

Test area	Validation goal
Startup and health	Service boots cleanly and <code>/health</code> returns expected liveness metadata.
API contract coverage	OpenAPI route inventory matches the expected endpoint contract (no undocumented drift).
Database connectivity and migrations	SQLAlchemy session round-trip works; Alembic chain applies cleanly from a fresh schema.
CSV ingestion and parsing	Papers, authors, topics, institutions, and optional citation edges import correctly; malformed rows are skipped with reporting.
Auth and authorization boundaries	Registration/login/token flows and ownership-scoped access control are enforced end-to-end.
Discovery and analytics endpoints	Filtering, pagination, and response-schema consistency across papers, authors, topics, and analytics routes.
Citation graph endpoints	Citation neighbourhood and directed BFS path correctness within the Leeds citation subgraph.
Embedding and similarity pipeline	Offline embedding generation progress and similarity ranking behaviour are validated.
Project, reading-list, and annotation workflows	Full CRUD lifecycle with strict JWT user isolation and not-found handling.
Recommendation ranking	Semantic, citation, and hybrid scoring modes (including weight handling) are validated with access-control checks.
MCP contract and mount	Public MCP tool registry and mounted HTTP MCP behaviour are verified.
Frontend build reproducibility	Frontend build output is validated and checked for committed bundle consistency.
PostgreSQL + pgvector integration	Cosine-distance query behaviour and similarity ranking are validated against a live <code>pgvector/pg17</code> service container in CI.

Unit tests run on in-memory SQLite (with a dialect-conditional vector path and Python cosine fallback), while PostgreSQL integration tests run in GitHub Actions against a `pgvector/pg17` service container. On every push and pull request, GitHub Actions enforces API/MCP contract checks, frontend build reproducibility, Ruff, full pytest, Alembic migration verification, and Docker smoke checks (app startup and MCP mount).

7 Deployment Workflow

Local development uses `docker compose -profile app` with `db` and `api`; container startup runs `alembic upgrade head` before launching Uvicorn. Cloud deployment uses the same Dockerfile on Railway, with `railway.toml` health checks targeting `/health`. The same FastAPI service hosts API routes, the frontend at `/app`, and the MCP HTTP mount at `/mcp`, keeping runtime behaviour aligned across local, CI, and deployed environments. Post-deploy validation follows the same checks used in CI: `/health` response verification, OpenAPI route-contract consistency, and MCP mount accessibility. The live service is then smoke-tested through key user flows (search, project creation, reading-list update, and recommendations) via the hosted frontend and API docs. This keeps runtime behaviour, schema state, and exposed interfaces aligned across local development, CI, and production. This deployment pathway is grounded in the documented migration and operations guidance for SQLAlchemy/Alembic, Railway, and Docker, reinforcing reproducibility across local and hosted environments.

8 Challenges and Current Limitations

Citation-graph construction from incomplete metadata. The OpenAlex CSV export provides aggregate `cited_by_count` values but not explicit citation edges, so graph traversal could not be implemented from ingestion alone. A dedicated `research-assistant-export-citation-graph` pipeline was therefore developed to query OpenAlex in batches, derive Leeds-to-Leeds edges, and persist resumable JSON checkpoints. This converted citation support from a static-data limitation into a

reproducible extraction workflow.

Data quality and ingestion robustness. The dataset included malformed rows and duplicate identifiers (for example repeated DOI/ORCID values). Strict ingestion constraints caused bulk-load failures, so the importer was refactored to skip invalid rows, continue processing, and report skipped counts. The trade-off is controlled tolerance over hard-fail behaviour, with transparent import summaries for auditability.

pgvector capability vs. test portability. Production similarity features rely on pgvector cosine search in PostgreSQL, while fast unit tests run on SQLite. Because SQLite does not support pgvector types/operators, a dialect-conditional vector path plus Python cosine fallback was required in tests. This increases implementation complexity, but keeps local and CI validation portable and deterministic.

Bounded citation-graph scope. Citation endpoints currently operate on a Leeds-to-Leeds subgraph, not the full OpenAlex graph. Some real shortest paths are therefore absent when they require non-Leeds intermediary papers. Endpoints and recommendation logic are documented and implemented as subgraph-aware to avoid overstating graph completeness.

9 Future Work and Research Extensions

Continuous OpenAlex synchronization. OpenAlex is continuously updated, while the current workflow starts from a downloaded snapshot plus enrichment jobs. A key extension is an incremental sync pipeline (scheduled via CI/cron) that performs idempotent upserts from OpenAlex API deltas, refreshes citation edges, and re-runs embeddings only for changed records. This would keep the deployed corpus current without full re-imports.

Generalised subset ingestion beyond a single institutional slice. Although this implementation uses a University of Leeds subset, the same pipeline can be parameterised to ingest other institution-, topic-, period-, or collaboration-based slices from the wider OpenAlex graph. This turns the system from a fixed coursework dataset into a reusable academic discovery platform.

Broader scholarly entity coverage. The current core focuses on works, authors, topics, and derived citation edges. Future schema extensions could incorporate additional OpenAlex entities (for example venues/sources, funders, and institutions as first-class nodes), enabling richer cross-entity analytics and recommendation signals.

Scalability and ranking quality. Further improvements include ANN indexes (HNSW/IVFFlat) for pgvector scale, async embedding refresh workers, and multi-hop citation-aware recommendation scoring to capture indirect influence paths.

Research reproducibility metadata. A provenance layer (snapshot date, API cursor/checkpoint IDs, pipeline version, embedding model/version) would strengthen scientific traceability and make comparative experiments across corpus versions reproducible.

10 Conclusion

The Research Assistant API delivers a full scholarly-discovery platform rather than a basic CRUD service. It combines resilient ingestion, explicit citation-graph extraction, semantic retrieval via embeddings, hybrid recommendation, authenticated project workflows, a browser-accessible interface, MCP integration, and live Railway deployment.

A key outcome is engineering reproducibility: pipelines are scripted, schema evolution is migration-driven, CI enforces quality gates on every push/pull request, and deployment uses the same containerised service model as local development. The resulting system demonstrates not only implementation breadth, but also architectural originality in turning incomplete bibliographic metadata into a continuously extensible research infrastructure.

References

1. Research Assistant API repository. <https://github.com/NathanWalash/research-assistant-api>
2. Priem, J., Piwowar, H., & Orr, R. (2022). OpenAlex: A fully-open index of the global research system. *arXiv*. <https://arxiv.org/abs/2205.01833>
3. FastAPI documentation. <https://fastapi.tiangolo.com/>
4. PostgreSQL 17 documentation. <https://www.postgresql.org/docs/17/>
5. pgvector: Open-source vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>
6. SQLAlchemy 2.0 documentation. <https://docs.sqlalchemy.org/>
7. Alembic documentation. <https://alembic.sqlalchemy.org/>
8. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP 2019*. <https://aclanthology.org/D19-1410/>
9. Sentence-Transformers documentation. <https://www.sbert.net/>
10. Sentence-Transformers model card: all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
11. Railway deployment documentation. <https://docs.railway.app/>
12. Docker documentation. <https://docs.docker.com/>
13. OpenAlex API documentation. <https://docs.openalex.org/>
14. PyJWT documentation. <https://pyjwt.readthedocs.io/>
15. React documentation. <https://react.dev/>
16. esbuild documentation. <https://esbuild.github.io/>
17. GitHub Actions documentation. <https://docs.github.com/actions>
18. Anthropic. (2024). Model Context Protocol specification. <https://modelcontextprotocol.io/>

Appendix A Generative AI Declaration and Reflection

A.1 Declared Tool Usage

Tool	Primary role	Usage and human oversight
ChatGPT	Problem framing and planning	Used at project start to convert the coursework brief into a concrete specification, delivery roadmap, and architecture trade-off matrix (framework, data model, deployment path). It was also used to explore OpenAlex ingestion strategy and scope reduction decisions. Outputs were treated as design hypotheses and manually validated against feasibility and marking requirements. Chat history link: https://chatgpt.com/share/69b1c560-6c78-8011-9d82-3171cbe07974 .
Codex	Agentic implementation support	Used during build to implement and refactor endpoints, schemas, services, and tests from the roadmap. Typical usage involved code generation followed by manual review, local verification, and CI validation (Ruff, pytest, migration checks, Docker smoke checks). No generated code was accepted without understanding and adapting it to repository constraints.
Claude	Long-form drafting and structure support	Used mainly after implementation for initial report/presentation drafting and narrative structure suggestions from the brief + repository context. Draft text was heavily edited, corrected, and aligned to actual code behaviour. Claude suggestions were used as a starting point, not final submission text.

A.2 Workflow Summary

AI was used as a staged engineering assistant across the full lifecycle:

1. **Pre-development (specification).** ChatGPT was used to turn the brief into a requirements checklist, milestone roadmap, and candidate architecture options.
2. **Data/pipeline design.** ChatGPT-assisted exploration helped identify the need for a separate citation-edge export pipeline because the base CSV did not provide explicit graph edges.
3. **Implementation (agentic coding).** Codex was used to accelerate route/service/test implementation, with manual repository-aware validation at each step.
4. **Issue handling.** For non-trivial design decisions, the process looped back to strategic prompting (usually ChatGPT), then returned to code-level execution and testing.
5. **Communication artifacts.** Claude was used for first-pass report/slide drafts, then manually revised for technical accuracy and alignment with the final repository state.

A.3 Representative Prompt Examples

Because some tools (for example Codex/Claude in this workflow) do not provide full exportable conversation logs, representative prompt patterns are documented below to evidence usage style:

- ChatGPT: "Given this coursework brief, produce a specification and roadmap with must/should/could priorities."
- ChatGPT: "Compare FastAPI+PostgreSQL + pgvector vs alternatives under coursework constraints."
- Codex: "Implement /papers/{id}/path/{target_id} BFS traversal over citations with tests and error handling."
- Codex: "Refactor ingestion to skip malformed rows, count skips, and keep batch import resilient."
- Codex: "Add CI contract gates for API/MCP plus Docker smoke tests and migration verification."
- Claude: "Draft technical report sections from coursework brief + repository link, then I will fact-check and edit."

A.4 Critical Reflection

A.4.1 Strategic Contribution During Scoping

The highest-value use of AI was in early-stage planning. Prompted against the brief, AI helped transform broad requirements into a staged specification, backlog, and delivery roadmap, which reduced ambiguity before coding began. It also accelerated comparison of architectural options (framework, database/vector strategy, API style), allowing decisions to be made earlier and with clearer trade-off awareness. This reduced rework later in implementation.

A.4.2 Influence on Data and Pipeline Design

AI support was especially useful for identifying dataset and pipeline implications. In particular, it helped surface that OpenAlex metadata alone was insufficient for graph traversal and that a separate citation-edge export process was needed. This directly informed the introduction of an offline, resumable citation export workflow, rather than trying to force graph behaviour from aggregate metadata fields.

A.4.3 Implementation Acceleration vs. Engineering Control

During implementation, AI tools were used to accelerate scaffold and routine code production (routes, schemas, fixtures, and refactors). However, speed gains were only reliable when paired with strict human control: every generated change was reviewed in-repository, adapted to project conventions, and validated through tests. In practice, AI improved throughput, but correctness still depended on deliberate engineering judgement.

A.4.4 Verification Protocol and Evidence

A consistent verification protocol was used before accepting AI-assisted output: local code inspection, targeted endpoint checks, test execution, and CI confirmation. This included linting, full pytest execution, migration verification, and container smoke checks in CI. The final suite size (127 tests across 42 modules, with PostgreSQL integration tests explicitly gated) demonstrates that validation was treated as a first-class activity rather than a final formality.

A.4.5 Limitations Encountered and Mitigation

The main limitation was context mismatch: AI sometimes produced generic or partially correct suggestions that did not fit the repository's current structure or constraints. Typical issues included over-general patterns, missing edge-case handling, and assumptions about unavailable runtime context. Mitigation required tighter prompts, explicit constraints, and evidence-based acceptance criteria (tests and runtime behaviour), which significantly reduced error propagation.

A.4.6 Authorship, Accountability, and Future Refinement

AI functioned as an accelerator, not an autonomous author. Final architecture, acceptance of code changes, and final report wording remained human decisions grounded in repository evidence. A future improvement would be to formalise this workflow further with a prompt/change log that links AI suggestions directly to accepted commits, tests, and rationale, strengthening traceability and academic accountability.

A.4.7 Security-Sensitive AI Usage

Security-relevant AI outputs were treated with stricter acceptance criteria than routine implementation suggestions. Recommendations touching authentication, token handling, access control, and deployment configuration were only accepted after repository-level verification against the implemented security model (JWT validation, user-ownership scoping, secret-management constraints, and default-disabled MCP exposure), followed by test and CI confirmation. Prompt hygiene was also enforced: no live credentials or sensitive deployment values were shared with AI tools, and examples were redacted or synthetic. This reduced both insecure-code risk and accidental information leakage during AI-assisted development.

Declaration. All GenAI tools used in this project are declared above. AI supported ideation, planning, implementation acceleration, debugging, and drafting; however, final design decisions, final code acceptance, and final report wording were authored through independent judgement and repository-grounded verification.